Howard University • Department of Mathematics

Setting Up Upstream in Git

Why `git push` keeps asking you for more

Dr. Moussa Doumbia • mdoumbia@howard.edu

You typed `git push`. Git said no.

```
$ git push
fatal: The current branch main has no upstream branch.
To push the current branch and set the remote as upstream, use

    git push --set-upstream origin main
```

What Git is actually complaining about

You told it to push. You never told it where.

Git refuses to guess, because guessing wrong means writing your work into somebody else's branch.

"Upstream" means two different things

1. Upstream *branch*

A link between your local branch and a branch on the remote.

Makes bare `git push` and `git pull` work.

This is the one causing your error.

2. Upstream *remote*

A second remote pointing at the *original* repo you forked from.

Lets you pull in the original author's later changes.

Only relevant if you clicked "Fork".

Same word, unrelated problems. We will cover both.

PART ONE

The Upstream Tracking Branch

Connecting a local branch to the remote

The mental model

Setting the upstream draws those two arrows once

After that, Git remembers. `git push` with no arguments knows the destination; `git pull` with no arguments knows the source.

Starting a brand-new repository

Six commands, in this order

```
git init # create the repo
git add . # stage everything
git commit -m "Initial commit"
git branch -M main # name the branch main
git remote add origin https://github.com/YOURNAME/REPO.git
git push -u origin main # push AND set upstream
```

The whole trick is one flag

`-u` is short for `--set-upstream`. You use it once, on the first push. From then on, `git push` and `git pull` need no arguments.

The branch already exists but isn't tracking

You forgot `-u`, or the branch was created some other way. Fix it without pushing:

```
git branch --set-upstream-to=origin/main main
```

Or just push again with the flag — same result:

```
git push -u origin main
```

Reading the command

`-set-upstream-to=origin/main` is the *target*.
The trailing `main` is the *local branch* being configured.

Verify it actually worked

```
git branch -vv
# * main 7f3a91c [origin/main] Initial commit
```

The `[origin/main]` in square brackets is the upstream. No brackets means no upstream.

```
git status
# On branch main
# Your branch is up to date with 'origin/main'.
```

Habit worth forming

Run `git status` after every setup step. It tells you the branch, the upstream, and what is staged — three answers for one short command.

Never type `-u` again

Git 2.37 and later can set the upstream automatically on first push:

```
git config --global push.autoSetupRemote true
```

Check your version first:

```
git --version
```

Worth doing once, on every machine you use

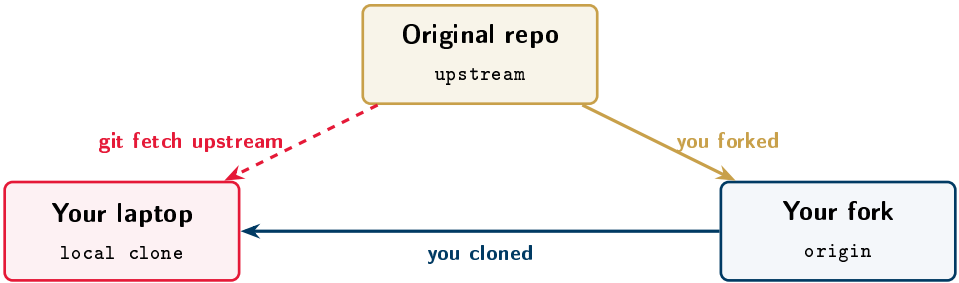
After this, a plain `git push` on a brand-new branch just works. The error from slide 2 never appears again.

PART TWO

The Upstream Remote

Staying in sync with a repo you forked

What a fork looks like



The dashed arrow is what you have to build yourself

Cloning gives you `origin` automatically. It does *not* give you a connection back to the original repo. You add that by hand.

Adding the upstream remote

```
git remote add upstream https://github.com/ORIGINAL/REPO.git
git remote -v
```

You should see four lines:

```
origin https://github.com/YOU/REPO.git (fetch)
origin https://github.com/YOU/REPO.git (push)
upstream https://github.com/ORIGINAL/REPO.git (fetch)
upstream https://github.com/ORIGINAL/REPO.git (push)
```

`upstream` is a convention, not a keyword

Git does not treat that name specially — you could call it `theirs`. Everyone uses `upstream`, so use it too.

Pulling in the original author's changes

```
git fetch upstream # download, change nothing yet
git merge upstream/main # bring them into your branch
git push origin main # update your fork on GitHub
```

Why `fetch` then `merge`, not `pull`?

`fetch` downloads without touching your files, so you can look before you leap:

```
git log HEAD..upstream/main -oneline
```

`pull` does both at once and gives you no chance to inspect.

Which one do you actually need?

Your situation	What you need
You created the repo yourself — course work, a portfolio, your own project	**Upstream branch** `git push -u origin main`
You clicked "Fork" on somebody else's repository	**Both** the branch *and* `git remote add upstream`
You cloned a repo you have write access to	**Neither** cloning sets the upstream for you

For this course, it is almost always the first row.

One gotcha before you push

GitHub stopped accepting passwords in 2021

If Git prompts for a password over HTTPS, your account password will not work. You need a **personal access token**.

1. GitHub → Settings → Developer settings
2. Personal access tokens → Generate new token
3. Tick the `repo` scope, set an expiry
4. Paste the token when Git asks for a *password*

```
git config --global credential.helper store # remember it
```

Copy the token immediately — GitHub shows it once.

Cheat sheet

SET IT UP

```
git push -u origin main

git branch --set-upstream-to=\
    origin/main main

git config --global \
    push.autoSetupRemote true
```

CHECK IT

```
git branch -vv
git status
git remote -v
```

FORK WORKFLOW

```
git remote add upstream URL
git fetch upstream
git merge upstream/main
```

If you remember one line

`git push -u origin main` — once, on the first push.

Questions?

Dr. Moussa Doumbia • mdoumbia@howard.edu

Office hours: ASB B 217